

Credit, Pricing & Payment System

Product Requirements Document • v1.0 • Draft, In Review • Author: Product

This document specifies how JobWhisper's credit system, subscription pricing, and payment processing should work together. It's written to be read start to finish by an engineer picking up the build — it explains the product decisions that are settled, and flags, explicitly, the ones that still need a call from the business before they can be built.

1 Overview

There are three layers here, and they need to stay conceptually separate even though they depend on each other:

- **Credits** — the currency users actually spend inside the product. Every metered action (writing a tailored resume, submitting a job application, running a live interview session) costs a fixed number of credits, deducted from a balance.
- **Pricing** — how a user gets credits and feature access in the first place: a monthly or annual subscription that includes a recurring credit allowance and a base set of features, optional recurring add-ons layered on top, and optional one-off top-ups.
- **Payments** — the real money behind all of it. This is the layer that doesn't exist yet. Every price, credit balance, and plan a user sees today is placeholder data — no payment has ever actually been processed. This document specifies what needs to be built to change that.

2 How Credits Work

2.1 What a credit is worth

- One credit is worth **\$0.40 USD**, fixed.
- Behind the scenes, every balance should be tracked in real currency (cents), never in "credits" as a native unit — credits are purely how the number is presented to the user. This matters for the payment build: money in, money out, and the ledger should all be cents; the credit figure is a conversion applied only when something is displayed.
- Credit amounts shown to a user are always a whole number, rounded up in magnitude — never rounded down in a way that could make a real deduction look like it cost "0 credits." A dollar figure shown next to a credit amount is always an exact conversion, never approximated or prefixed with "~".

2.2 What each feature costs to use

Feature	What triggers a charge	Rate
Resume Builder	One prompt/message sent to the AI	1 credit (\$0.40) / message
Auto Apply	One successful job application submitted	3 credits (\$1.20) / application

Feature	What triggers a charge	Rate
Interview Prep	Per minute of a live practice session	2 credits (\$0.80) / min
Interview Copilot	Per minute of a live session	2 credits (\$0.80) / min
Coding Copilot	Per minute of a live session	2 credits (\$0.80) / min
Meeting Copilot	Per minute of a live session	2 credits (\$0.80) / min
ATS Scoring	Click "Score Resume"	Free
AI Suggester	Rewrite a phrase or statement	Free

A separate, related initiative — a desktop version of the interview copilot sold under different pricing — currently plans to bill live sessions by the hour (1 credit per hour or fraction of an hour) instead of per minute, at different subscription prices (\$49/\$79 instead of \$100/\$200). If both are meant to be the same underlying product, this needs to be reconciled into one metering model and one price before either ships — see the open questions in §8.

2.3 How a balance gets credited

There are three ways a wallet balance increases:

- 1 **Monthly subscription renewal.** On each successful billing cycle, the wallet refills to the plan's included credit allowance (e.g. a Pro subscriber's balance refreshes to 100 credits). Whether unused credits carry over into the next cycle, or the balance simply resets to the plan allowance regardless of what's left, is not yet decided — see §8.
- 2 **A one-off top-up.** A user can add credits directly, in a preset or custom amount, at any time. Once added, top-up credits stay on the account until spent — they are not reset or clawed back at the next renewal, unlike the subscription's included allowance.
- 3 **A referral bonus.** A fixed credit grant when someone the user referred subscribes. The exact amount, eligibility rules, and whether the bonus is reciprocal are not yet specified — see §8.

2.4 Free trial balance

A user with no active subscription still gets **5 free credits every month**, on a rolling basis. Before this can be built against real time, product needs to confirm the reset anchor: does it reset on the calendar month, or 30 days from the date the person signed up?

3 Subscription Plans & Pricing

3.1 Plan tiers

Tier	Price	Surface	Interview	Coding	Meeting	Credits/mo
Starter	\$20/mo · \$192/yr	Web only	Yes	No	No	20

Tier	Price	Surface	Interview	Coding	Meeting	Credits/mo
Pro	\$100/mo · \$960/yr	Web + Desktop	Yes	Yes	No	100
Premium	\$200/mo · \$1,920/yr	Web + Desktop	Yes	Yes	Yes	200

Resume Builder and Auto Apply are deliberately excluded from every tier, including Premium — they're sold separately as add-ons (§3.2). This is a strategic choice: rather than capping revenue per user at the top subscription tier, Resume Builder and Auto Apply each become their own upsell surface with room to keep growing account value well past the Premium price point.

3.2 Add-ons

Add-ons require an active subscription of any tier — they aren't sold to accounts without one. Each is billed as its own recurring charge and can be cancelled independently of the base plan.

Add-on	Price	Further unlock available inside it
Resume Builder	\$15/mo	AI Suggestions & Premium Templates — price not yet set, unlocked only once Resume Builder itself is active
Auto Apply	\$40/mo	Full-Auto Mode — +\$10/mo, removes the manual step of picking which jobs to apply to, unlocked only once Auto Apply itself is active

At checkout, when a user subscribes to any tier, offer Auto Apply and/or Resume Builder as an order bump before the purchase completes — unchecked by default, cart total updates live if the user adds one, and it's dismissible without blocking the base subscription. The further unlocks inside each add-on (Full-Auto Mode, AI Suggestions) are not offered at this moment — they should be surfaced later, inside the add-on's own product experience, once the user is actually using it.

3.3 What each tier and add-on unlocks

Capability	Starter	Pro	Premium	Resume add-on	Auto Apply add-on
Interview Copilot (web)	✓	✓	✓	—	—
Interview Copilot (desktop)	—	✓	✓	—	—
Coding Copilot	—	✓	✓	—	—
Meeting Copilot	—	—	✓	—	—
Resume Builder	—	—	—	✓	—
— AI Suggestions / Templates	—	—	—	further unlock	—
Auto Apply (semi-manual)	—	—	—	—	✓
— Full-Auto mode	—	—	—	—	further unlock

4 Payment System

This is the part of the spec that's genuinely new — none of it is built yet. It describes how real money should move through the product, end to end.

4.1 Which payment provider

Recommendation: Stripe, specifically its subscription billing, hosted checkout, and self-serve customer portal. The reasoning: it keeps card details off our own servers entirely, and its data model handles "one base plan plus several independent add-ons" cleanly, without custom work to calculate what a user owes when they change something mid-cycle. This is a recommendation for sign-off, not a decision already made — flag it if there's a reason to use something else.

4.2 How money and credits relate

The payment provider only ever needs to know about real currency — subscription prices, add-on prices, top-up amounts, all in dollars and cents. **It should never need to know what a "credit" is.** Credits are purely something we compute and display after a payment succeeds; the actual record of what was paid and what's owed always lives in dollars and cents.

4.3 The checkout experiences to build

- **New subscription, with the add-on order bump.** A user picks a tier and optionally adds Resume Builder and/or Auto Apply before paying. One checkout, one payment, everything they picked included.
- **A one-off credit top-up.** A user picks a preset or custom amount and pays for it directly — available to subscribers and non-subscribers alike.
- **Adding or removing an add-on mid-cycle.** A subscriber turns on Auto Apply partway through their billing cycle; they should be charged a prorated amount for the remainder of that cycle, and the feature should unlock the moment payment succeeds — never before payment is confirmed, no matter what the interface shows optimistically.
- **Upgrading or downgrading the base plan.** An upgrade should take effect, and charge the prorated difference, immediately. A downgrade should wait until the current billing period ends — a user shouldn't lose something they already paid for this cycle.
- **Cancelling.** Default to letting the user keep access through the end of the period they already paid for, rather than cutting them off the moment they cancel. Whether cancelling the base plan should also cancel any add-ons riding on it, or let them keep running independently, isn't decided yet — see §8.

4.4 Keeping our records honest

The payment provider's record of what was actually paid is always the source of truth. Our own database mirrors it — it never leads it. This is the only safe way to handle a user closing the tab mid-payment, a charge that succeeds a few seconds later than expected, or a bug on our end that thinks a payment succeeded when it didn't.

In practice, this means every meaningful change of state — a new subscription starting, a renewal succeeding, a payment failing, a plan changing, a cancellation taking effect — should be driven by a confirmed notification from the payment provider, not by trusting what the checkout page tells us happened. Concretely, at minimum the system needs to react to: a checkout completing, a renewal invoice being paid, a renewal invoice failing, a

subscription being changed, a subscription ending, and a payment being disputed by the cardholder's bank. Every one of these needs to be handled in a way that's safe to receive twice — a duplicate notification of the same event should never double-charge a wallet or provision the same subscription a second time.

4.5 What unlocks what

A user's current plan, active add-ons, and credit balance should be resolved from their real subscription state, not from a fixed, hand-written record. Nothing about how this is shown to the user needs to change — a plan, an add-on, and a wallet balance are already well-defined concepts in the product — only where that information actually comes from changes, from something fixed to something computed live from what was actually paid for.

4.6 The credit ledger, and holding credits during a live session

- Every event that changes a credit balance — a renewal grant, a top-up, a referral bonus, a feature charge — should be its own permanent record, never a single number that gets overwritten. The current balance is always the sum of that history, which is what makes a user's transaction history trustworthy rather than just a number that might be right.
- A live session (an interview, a coding round, a meeting) bills by the minute while it's still running. To stop a session from running the balance past zero, or two sessions from racing to spend the same last credit, the system needs to place a hold on credits at the start of a session and settle it as the session progresses or ends, releasing anything unused.
- Every write to the ledger that's triggered by a payment-provider notification needs to be tied to that notification's unique ID, so a duplicate notification can never be applied twice.

4.7 When a payment fails

A failed renewal charge should not immediately lock the account. Give it a grace period — most payment providers already retry a failed card automatically over roughly two weeks, and that's a reasonable default window to use unless product wants something different. During that window, the user should clearly see that a payment failed and be prompted to update their card, without losing access in the meantime. If every retry fails and the subscription is ultimately cancelled for non-payment, the account should fall back to the free trial balance (\$2.4), not to zero access.

4.8 Refunds and disputes

No policy exists yet — this needs a decision from product and finance before it can be built. Once there is one: a refund should reverse the matching credit grant rather than leaving a wallet funded by money that was given back, and a disputed charge should flag the account for a person to review rather than being auto-revoked, since a false dispute shouldn't instantly lock out a legitimate customer.

4.9 Security

- Card details should never touch our own servers or frontend at all — routing every payment through the provider's own hosted checkout and account-management pages is what makes this true, and keeps our compliance burden to the minimum tier.
- Every notification received from the payment provider must be verified as genuinely from them before it's acted on.

- No tax is collected anywhere in the product today. If JobWhisper needs to charge sales tax or VAT in any region, that needs to be scoped separately — not assumed in or out.
- Pricing today is USD only, with no stated requirement for other currencies.

5 Data the System Needs to Track

In plain terms, not a schema — the engineer building this will design the actual structure, but these are the concepts that need a home somewhere:

- **Every user's account**, linked to their record with the payment provider.
- **Every subscription**, its status (active, past due, cancelled, etc.), and when its current paid period ends.
- **Every individual thing being paid for on that subscription** — the base plan, and each add-on — since they can each change independently.
- **The credit ledger** — one permanent row per balance-changing event, with where it came from and, where relevant, the ID of the payment-provider notification that caused it, so nothing can be double-counted.
- **Credit holds** — amounts reserved for a session that's still in progress, separate from the settled ledger until the session ends.

6 What "Working Correctly" Means

- **Nothing gets missed or double-counted.** Every payment notification is processed exactly once in effect, even if it's delivered more than once, and the system should raise an alarm if processing ever falls meaningfully behind.
- **Our records and the payment provider's records never quietly drift apart.** A regular check comparing the two should catch it immediately if they ever do.
- **Checking what a user has access to is fast.** This gets checked on nearly every page load, so it needs to be quick even though the real source of truth lives with the payment provider.
- **Every credit and every access change can be explained.** A support agent — or the user themselves — should always be able to see exactly why a balance or an entitlement changed.

7 How We'll Know It's Working

- Checkout completion rate — tracked separately for new subscriptions, add-on purchases, and credit top-ups, since they're different funnels with different friction.
- How much churn is involuntary (failed payments) versus voluntary (the user chose to cancel) — these need different fixes.
- How often a payment notification is handled successfully the first time, without needing a retry.
- How often our records and the payment provider's records are found to disagree — the target is zero, and this needs to actually be measured to know it's true.
- How long it takes from a successful payment to the feature actually being unlocked for the user — this should be close to instant.

8 Open Questions — Needs a Decision Before Build

- 1 What should the price be for the Resume Builder's further unlock (AI Suggestions & Premium Templates)?
- 2 Do unused monthly subscription credits carry over into the next cycle, or does the balance simply reset to the plan's allowance regardless of what's left?
- 3 Should add-ons ever be offered at an annual price, or are they monthly only?
- 4 How should this plan's Pro/Premium pricing and per-minute billing be reconciled with the separate desktop-app initiative pricing the same tiers differently, with per-hour billing? One product or two? One price or two?
- 5 If a user cancels or downgrades their base plan, should any add-ons they have automatically cancel with it, or keep running independently?
- 6 What is the referral bonus amount, and what are the eligibility rules?
- 7 Is the payment-provider recommendation (§4.1) confirmed, or does a specific market or requirement call for something else?
- 8 How long should the grace period be for a failed payment — the provider's own default retry window, or something product wants to specify directly?
- 9 What is the refund and dispute policy? Nothing is defined yet.
- 10 Is charging sales tax / VAT in scope for the first version, or explicitly deferred?
- 11 Is support for currencies other than USD confirmed out of scope, or has it just not come up yet?
- 12 Should a plan downgrade take effect at the end of the current billing period (recommended above) or immediately?

9 Suggested Build Order

- 1 Get sign-off on the pricing and credit-allowance numbers still marked open above — several downstream steps are blocked on these.
- 2 Set up the payment provider account and the full catalog of products and prices for every tier, add-on, and further unlock.
- 3 Build the backend records described in §5, and the endpoint that receives and verifies notifications from the payment provider.
- 4 Implement the handlers for each of the events listed in §4.4, writing to those records, with duplicate-safe processing.
- 5 Wire up the new-subscription checkout, including the add-on order bump.
- 6 Wire up the one-off credit top-up purchase flow.
- 7 Wire up adding/removing an add-on and upgrading/downgrading a plan against real, prorated charges.
- 8 Build the "payment failed, please update your card" state described in §4.7 — there's no equivalent of this in the product today.
- 9 Build the regular reconciliation check described in §6, plus basic internal visibility into a user's ledger and subscription state, for support to use when debugging.

10 Refunds, disputes, tax, and whichever of the remaining open questions get resolved last.